

NT-C-001 | Python Full Stack | Course Knowledge Profile

Identity, purpose, prerequisites, scope and boundaries

Category	Programming & Full Stack
Target learners	Graduates, job seekers and working professionals seeking web-application roles.
Prerequisites	Basic computer use and logical thinking; prior coding is helpful but not mandatory.
Approved duration	Approved working range: 20-24 weeks / 240-300 guided hours.
Final outcome	Build, test and deploy a database-backed web application with authentication, REST APIs and a modern user interface.
Assessment profile	Programming/Platform

Approved tools / platforms

Python, Django, Django REST Framework, HTML, CSS, JavaScript, React, SQL, Git; database and hosting stack to be confirmed.

Knowledge boundaries

Include	Quality rule	Exclude / escalate
The eight approved modules, four sections, named tools, projects and mapped entry-level roles.	Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.	Do not treat copied code or syntax recall as proof of implementation skill.
Current product/platform procedures only when version and date are known.	Use primary documentation and record the source version.	Do not invent current commands, prices, tax rates, policies or certification status.
Evidence-based career guidance.	Cite tests, projects, capstone, viva and verified resume evidence.	No guaranteed job, placement, salary, interview or employer claim.

LLM answer pattern: identify the course and version, answer from approved records, cite record IDs, distinguish verified facts from guidance, and state any missing evidence.

NT-C-001 | Python Full Stack | NT-C-001-S01 Deep Knowledge

Modules 1-2 | objectives, concepts, evidence and remediation

Section competency	Syntax, data types, control flow, functions; OOP, files, exceptions, testing, Git
Completion evidence	Working command-line programs; Tested Python package
Section weight	20% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 1 - Python foundations

Objective: Syntax, data types, control flow, functions

Observable evidence: Working command-line programs

Concept	Approved knowledge statement	Application behaviour	Evidence
Syntax	The formal rules used to write valid instructions in a programming language. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when syntax is appropriate; apply it in a Python Full Stack task; verify the result.	Working command-line programs
data types	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when data types is appropriate; apply it in a Python Full Stack task; verify the result.	Working command-line programs
control flow	The order in which conditions, loops and branches execute. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when control flow is appropriate; apply it in a Python Full Stack task; verify the result.	Working command-line programs
functions	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when functions is appropriate; apply it in a Python Full Stack task; verify the result.	Working command-line programs

Module 2 - Professional Python

Objective: OOP, files, exceptions, testing, Git

Observable evidence: Tested Python package

Concept	Approved knowledge statement	Application behaviour	Evidence
OOP	A design approach that organises behaviour and data around objects and clear responsibilities. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when oop is appropriate; apply it in a Python Full Stack task; verify the result.	Tested Python package
files	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when files is appropriate; apply it in a Python Full Stack task; verify the result.	Tested Python package
exceptions	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when exceptions is appropriate; apply it in a Python Full Stack task; verify the result.	Tested Python package
testing	Systematic verification that expected behaviour and important failure cases work correctly. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when testing is appropriate; apply it in a Python Full Stack task; verify the result.	Tested Python package
Git	Distributed version control for tracking and reviewing source changes. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when git is appropriate; apply it in a Python Full Stack task; verify the result.	Tested Python package

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Working command-line programs; Tested Python package, return to Modules 1-2, complete one guided practice and then use a new equivalent test form.

NT-C-001 | Python Full Stack | NT-C-001-S02 Deep Knowledge

Modules 3-4 | objectives, concepts, evidence and remediation

Section competency	SQL, schema design, CRUD, joins; HTML, CSS, responsive layouts, JavaScript DOM
Completion evidence	Relational database exercise; Responsive front-end
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 3 - Database skills

Objective: SQL, schema design, CRUD, joins

Observable evidence: Relational database exercise

Concept	Approved knowledge statement	Application behaviour	Evidence
SQL	A language for defining, querying and controlling relational data. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when sql is appropriate; apply it in a Python Full Stack task; verify the result.	Relational database exercise
schema design	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when schema design is appropriate; apply it in a Python Full Stack task; verify the result.	Relational database exercise
CRUD	The create, read, update and delete lifecycle for persistent records. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when crud is appropriate; apply it in a Python Full Stack task; verify the result.	Relational database exercise
joins	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when joins is appropriate; apply it in a Python Full Stack task; verify the result.	Relational database exercise

Module 4 - Web foundation

Objective: HTML, CSS, responsive layouts, JavaScript DOM

Observable evidence: Responsive front-end

Concept	Approved knowledge statement	Application behaviour	Evidence
HTML	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when html is appropriate; apply it in a Python Full Stack task; verify the result.	Responsive front-end
CSS	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when css is appropriate; apply it in a Python Full Stack task; verify the result.	Responsive front-end
responsive layouts	A layout that adapts to different screen sizes and input methods. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when responsive layouts is appropriate; apply it in a Python Full Stack task; verify the result.	Responsive front-end
JavaScript DOM	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when javascript dom is appropriate; apply it in a Python Full Stack task; verify the result.	Responsive front-end

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Relational database exercise; Responsive front-end, return to Modules 3-4, complete one guided practice and then use a new equivalent test form.

NT-C-001 | Python Full Stack | NT-C-001-S03 Deep Knowledge

Modules 5-6 | objectives, concepts, evidence and remediation

Section competency	URLs, requests, views, templates, forms; ORM, authentication, admin, REST APIs
Completion evidence	Server-rendered CRUD app; Secured API and data model
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 5 - Django core

Objective: URLs, requests, views, templates, forms

Observable evidence: Server-rendered CRUD app

Concept	Approved knowledge statement	Application behaviour	Evidence
URLs	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when urls is appropriate; apply it in a Python Full Stack task; verify the result.	Server-rendered CRUD app
requests	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when requests is appropriate; apply it in a Python Full Stack task; verify the result.	Server-rendered CRUD app
views	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when views is appropriate; apply it in a Python Full Stack task; verify the result.	Server-rendered CRUD app
templates	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when templates is appropriate; apply it in a Python Full Stack task; verify the result.	Server-rendered CRUD app
forms	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when forms is appropriate; apply it in a Python Full Stack task; verify the result.	Server-rendered CRUD app

Module 6 - Backend application

Objective: ORM, authentication, admin, REST APIs

Observable evidence: Secured API and data model

Concept	Approved knowledge statement	Application behaviour	Evidence
ORM	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when orm is appropriate; apply it in a Python Full Stack task; verify the result.	Secured API and data model
authentication	Verification of identity. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when authentication is appropriate; apply it in a Python Full Stack task; verify the result.	Secured API and data model
admin	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when admin is appropriate; apply it in a Python Full Stack task; verify the result.	Secured API and data model
REST APIs	An http-oriented interface style using resources, methods and predictable responses. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when rest apis is appropriate; apply it in a Python Full Stack task; verify the result.	Secured API and data model

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Server-rendered CRUD app; Secured API and data model, return to Modules 5-6, complete one guided practice and then use a new equivalent test form.

NT-C-001 | Python Full Stack | NT-C-001-S04 Deep Knowledge

Modules 7-8 | objectives, concepts, evidence and remediation

Section competency	React components, state, routing, API integration; Testing, security basics, deployment, documentation
Completion evidence	API-connected interface; Deployed capstone
Section weight	30% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 7 - Modern front end

Objective: React components, state, routing, API integration

Observable evidence: API-connected interface

Concept	Approved knowledge statement	Application behaviour	Evidence
React components	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when react components is appropriate; apply it in a Python Full Stack task; verify the result.	API-connected interface
state	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when state is appropriate; apply it in a Python Full Stack task; verify the result.	API-connected interface
routing	Selection and forwarding of network paths between destinations. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when routing is appropriate; apply it in a Python Full Stack task; verify the result.	API-connected interface
API integration	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when api integration is appropriate; apply it in a Python Full Stack task; verify the result.	API-connected interface

Module 8 - Quality & deployment

Objective: Testing, security basics, deployment, documentation

Observable evidence: Deployed capstone

Concept	Approved knowledge statement	Application behaviour	Evidence
Testing	Systematic verification that expected behaviour and important failure cases work correctly. In Python Full Stack, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when testing is appropriate; apply it in a Python Full Stack task; verify the result.	Deployed capstone
security basics	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when security basics is appropriate; apply it in a Python Full Stack task; verify the result.	Deployed capstone
deployment	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when deployment is appropriate; apply it in a Python Full Stack task; verify the result.	Deployed capstone
documentation	A defined competency within Python Full Stack. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when documentation is appropriate; apply it in a Python Full Stack task; verify the result.	Deployed capstone

Remediation: If the learner cannot explain the purpose, complete the stated task or provide API-connected interface; Deployed capstone, return to Modules 7-8, complete one guided practice and then use a new equivalent test form.

NT-C-001 | Python Full Stack | Skill Ontology & Dependency Map

Atomic skills, learning order and evidence strength

Skill ID	Competency	Depends on	Evidence	Type
NT-C-001-SK01	Python foundations	Prerequisite foundation	Working command-line programs	Critical
NT-C-001-SK02	Professional Python	M01 plus earlier foundations	Tested Python package	Critical
NT-C-001-SK03	Database skills	M02 plus earlier foundations	Relational database exercise	Supporting
NT-C-001-SK04	Web foundation	M03 plus earlier foundations	Responsive front-end	Supporting
NT-C-001-SK05	Django core	M04 plus earlier foundations	Server-rendered CRUD app	Supporting
NT-C-001-SK06	Backend application	M05 plus earlier foundations	Secured API and data model	Critical
NT-C-001-SK07	Modern front end	M06 plus earlier foundations	API-connected interface	Supporting
NT-C-001-SK08	Quality & deployment	M07 plus earlier foundations	Deployed capstone	Critical

Evidence-strength hierarchy

Strength	Evidence source	Use
High	Trainer-observed practical, viva, working source/project files and reproducible output.	May support verified mastery when rubric threshold is met.
Medium	Scored mock tests, reviewed assignments and documented portfolio artifacts.	Supports mastery with corroboration.
Low	Resume claim, self-report, attendance or video completion without demonstration.	May trigger verification; cannot alone support Apply Now.

CareerTours must not treat attendance, time spent or content opened as skill mastery.

NT-C-001 | Python Full Stack | Assessment Knowledge & Sample Items

Non-live examples for LLM training and evaluation

Sample ID	Type	Question	Approved answer / rubric anchor
NT-C-001-S01-EX01	Evidence/application	What evidence best demonstrates the combined competency of Python foundations and Professional Python?	The learner should provide Working command-line programs; Tested Python package and explain how the work applies Syntax, data types, control flow, functions; OOP, files, exceptions, testing, Git. Recall alone is insufficient.
NT-C-001-S01-EX02	Scenario/remediation	A learner reports completing Professional Python but cannot produce Tested Python package. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-001-S01, request Tested Python package, and allow a new equivalent test only after remediation.
NT-C-001-S02-EX01	Evidence/application	What evidence best demonstrates the combined competency of Database skills and Web foundation?	The learner should provide Relational database exercise; Responsive front-end and explain how the work applies SQL, schema design, CRUD, joins; HTML, CSS, responsive layouts, JavaScript DOM. Recall alone is insufficient.
NT-C-001-S02-EX02	Scenario/remediation	A learner reports completing Web foundation but cannot produce Responsive front-end. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-001-S02, request Responsive front-end, and allow a new equivalent test only after remediation.
NT-C-001-S03-EX01	Evidence/application	What evidence best demonstrates the combined competency of Django core and Backend application?	The learner should provide Server-rendered CRUD app; Secured API and data model and explain how the work applies URLs, requests, views, templates, forms; ORM, authentication, admin, REST APIs. Recall alone is insufficient.
NT-C-001-S03-EX02	Scenario/remediation	A learner reports completing Backend application but cannot produce Secured API and data model. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-001-S03, request Secured API and data model, and allow a new equivalent test only after remediation.
NT-C-001-S04-EX01	Evidence/application	What evidence best demonstrates the combined competency of Modern front end and Quality & deployment?	The learner should provide API-connected interface; Deployed capstone and explain how the work applies React components, state, routing, API integration; Testing, security basics, deployment, documentation. Recall alone is insufficient.
NT-C-001-S04-EX02	Scenario/remediation	A learner reports completing Quality & deployment but cannot produce Deployed capstone. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-001-S04, request Deployed capstone, and allow a new equivalent test only after remediation.

Live bank rule: Generate at least three times the live quantity for each section using the approved Programming/Platform blueprint. Human-review every scored item before initial production release.

NT-C-001 | Python Full Stack | Labs, Projects & Scoring Rubrics

Practical proof of learning and ownership

Level	Approved project	Skills evidenced	Required deliverables
Mini 1	Expense or task tracker	Python, SQL, CRUD	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Mini 2	Student enquiry manager	Django, auth, forms, database	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Capstone	Admissions/LMS/CRM-style web application	API, React, testing, deployment	Brief; working/source file or reproducible environment; output; validation notes; reflection.

Standard project rubric

Dimension	Weight	What earns credit
Problem framing	15%	Clear objective, user/business need, assumptions and success criteria.
Technical/design accuracy	25%	Correct methods, appropriate tools and sound decisions.
Completeness	15%	Required features, assets, data or workflow are present.
Testing and validation	15%	Important normal, edge, quality or failure cases are checked.
Evidence and reproducibility	15%	Source/working files, setup, inputs, outputs and version information are available.
Documentation	5%	Concise structure, instructions and limitations.
Viva / ownership	10%	Learner can explain decisions, demonstrate work and respond to a change request.

Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.

NT-C-001 | Python Full Stack | Career Roles, Evidence & Gap Actions

Explainable top-three occupation matching

Rank	Occupation	Default weighted skills	Minimum evidence	Gap action
1	Junior Python Developer	Python 25% [critical]; OOP 25% [critical]; SQL 25%; Git 25%	Tested application repository	If a critical skill is below 60 or tested application repository is missing, return Improve First or Needs Evidence with the linked module/project.
2	Junior Django Developer	Django 25% [critical]; ORM 25% [critical]; auth 25%; APIs 25%	Deployed Django project	If a critical skill is below 60 or deployed django project is missing, return Improve First or Needs Evidence with the linked module/project.
3	Full Stack Web Developer - Python	Front end 25% [critical]; backend 25% [critical]; database 25%; deployment 25%	End-to-end capstone	If a critical skill is below 60 or end-to-end capstone is missing, return Improve First or Needs Evidence with the linked module/project.

Course readiness	45% section mocks + 20% mini projects + 25% capstone + 10% viva.
Role readiness	45% role-relevant mastery + 25% projects/capstone + 20% aligned mocks + 10% verified resume evidence.
Evidence confidence	Show separately; Apply Now requires at least 70.
Resume extraction	Use only relevant skills, projects, education, experience and portfolio evidence; minimise personal data before external model processing.

A student may be Apply Now for one mapped occupation and Improve First for another. Return independent evidence and gaps for each role.

NT-C-001 | Python Full Stack | FAQs, Misconceptions & LLM Response Pack

Course-specific retrieval and response patterns

Approved FAQ	Answer
Who is Python Full Stack for?	Graduates, job seekers and working professionals seeking web-application roles.
What should I know before starting?	Basic computer use and logical thinking; prior coding is helpful but not mandatory.
How long is the approved learning journey?	Approved working range: 20-24 weeks / 240-300 guided hours.
What will I be able to demonstrate?	Build, test and deploy a database-backed web application with authentication, REST APIs and a modern user interface.
Does completion guarantee a job?	No. Completion and role readiness are evidence-based guidance; employment, placement, salary and employer selection are not guaranteed.
What if I fail a section?	CareerTours identifies the failed skill tags and module, assigns targeted practice, and permits an equivalent retest after remediation.

Misconception	Approved correction
Attendance proves mastery	False - require tests and observable practical/project evidence.
One total score proves every role fit	False - calculate each occupation using its weighted skills and evidence.
The LLM can add missing details	False - retrieve approved records or state that evidence is insufficient.
Course completion guarantees employment	False - never claim guaranteed job, placement, salary or interview.
Python Full Stack knowledge never changes	False - version-sensitive tools, laws, taxes, security and platform procedures require review.

Retrieval metadata

```
course_code=NT-C-001; corpus_version=3.0;
record_types=course_profile,module,section,skill,assessment_sample,project,occupation_mapping,faq,misconception;
approval_status=approved; effective_date=2026-08-12
```